Eventhub_frontend

SOLID, Interface et tests unitaires

Damien JOSSE

Objectif des features

- Inscriptions d'un nouvel utilisateur
- Connexion d'un utilisateur existant
- Formulaire du profil (update des informations de l'utilisateur

Architecture du projet

- Schéma de l'architecture en couches = Mixage entre Feature Base et Domain Base
- Organisation :
  - Components/ (exemple fichier : Login.tsx)
  - Tests/ (exemple fichier : Login.test.tsx)
  - Hook/ (exemple fichier : use.login.ts)
  - Le store/ avec Redux
    - Auth/ pour auth.slice.ts

```
├── assets
└── modules
    ├── app
    │   └── components
    ├── checkPassword
    │   ├── components
    │   └── tests
    ├── login
    │   ├── components
    │   ├── hook
    │   └── tests
    ├── model
    ├── profile
    │   ├── components
    │   └── tests
    ├── register
    │   ├── components
    │   └── tests
    ├── store
    │   └── auth
    ├── testing
    └── updateProfile
        └── components
```

C'est quoi SOLID ?

S	Single Responsibility Principle (SRP)	Une classe / un composant / un fichier doit avoir une seule responsabilité.
O	Open / Closed Principle (OCP)	Ouvert à l'extension, fermé à la modification.
L	Liskov Substitution Principle (LSP)	Un type doit pouvoir être remplacé par un autre sans casser le comportement.
I	Interface Segregation Principle (ISP)	Ne pas forcer un composant à dépendre de choses qu'il n'utilise pas.
D	Dependency Inversion Principle (DIP)	Dépendre d'abstractions, pas d'implémentations.

Application des principes SOLID

S – Single Responsibility Principle (SRP)

- Une classe / un composant / un fichier doit avoir une seule responsabilité.

CheckPassword.tsx U

3WA > 2025 > Projet_Final > event-hub > eventhub_frontend > src > modules > checkPassword

```
import * as React from "react";  8.5k (gzipped: 3.3k)

Windsurf: Refactor | Explain
interface CheckPasswordProps {
    password: string;
    onValidityChange: (isValid: boolean) => void;
}

Windsurf: Refactor | Explain | Generate JSDoc |
export const CheckPassword: React.FC<CheckPasswordProps> = ({
    password,
    onValidityChange,
    }) => {
    const rules = {
        length: password.length >= 12,
        uppercase: /[A-Z]/.test(password),
        lowercase: /[a-z]/.test(password),
        number: /[0-9]/.test(password),
        special: /[^A-Za-z0-9]/.test(password),
    };

    const isValid = Object.values(rules).every(Boolean);

    React.useEffect(() => {
        onValidityChange(isValid);
    }, [isValid, onValidityChange]);

    Windsurf: Refactor | Explain | Generate JSDoc |
    function renderRule(label: string, valid: boolean) {
        return (
        <li style={{ color: valid ? "green" : "red" }}>
            {valid ? "✓" : "✗"} {label}
        </li>
        );
    }
```

Application des principes SOLID

L- Liskov Substitution Principle (LSP)

- Un type doit pouvoir être remplacé par un autre sans casser le comportement.

CheckPassword.tsx U

3WA > 2025 > Projet_Final > event-hub > eventhub_frontend > src > modules > checkPassword

```
1   import * as React from "react";  8.5k (gzipped: 3.3k)
2
    Windsurf: Refactor | Explain
3   interface CheckPasswordProps {
4       password: string;
5       onValidityChange: (isValid: boolean) => void;
6   }
7
    Windsurf: Refactor | Explain | Generate JSDoc | X
8   export const CheckPassword: React.FC<CheckPasswordProps> = ({
9       password,
10      onValidityChange,
11      }) => {
12      const rules = {
13          length: password.length >= 12,
14          uppercase: /[A-Z]/.test(password),
15          lowercase: /[a-z]/.test(password),
16          number: /[0-9]/.test(password),
17          special: /[^A-Za-z0-9]/.test(password),
18      };
19
20      const isValid = Object.values(rules).every(Boolean);
21
22      React.useEffect(() => {
23          onValidityChange(isValid);
24      }, [isValid, onValidityChange]);
25
        Windsurf: Refactor | Explain | Generate JSDoc | X
26      function renderRule(label: string, valid: boolean) {
27          return (
28          <li style={{ color: valid ? "green" : "red" }}>
29              {valid ? "✓" : "✗"} {label}
30          </li>
31          );
32      }
```

Application des principes SOLID

I — Interface Segregation Principle (ISP)

- Ne pas forcer un composant à dépendre de choses qu'il n'utilise pas.

UpdateProfile.tsx U

3WA > 2025 > Projet_Final > event-hub > eventhub_frontend > src > modules > updatePr

```
1   import { useState } from "react";  4.6k (gzipped: 1.9k)
2   import { useAppDispatch } from "../../store/store";
3   import { updateUser } from "../../store/auth/auth.slice";
4   import type { User } from "../../model/user";
5
    Windsurf: Refactor | Explain
6   interface Props {
7       user: User;
8       onDone: () => void;
9   }
10
```

Les composants React

```react.fc
export const Register: React.FC = () => {
    const [form, setForm] = useState<RegisterForm>({
        name: "",
        email: "",
        password: "",
    });

    const [isPasswordValid, setIsPasswordValid] = useState(false);

    const isFormValid =
        form.name.trim() !== "" &&
        form.email.trim() !== "" &&
        isPasswordValid;

    Windsurf: Refactor | Explain | Generate JSDoc | X
    function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
        e.preventDefault();

        if (!isFormValid) return;

        console.log("Utilisateur inscrit :", form);
    }

    Windsurf: Refactor | Explain | Generate JSDoc | X
    function handleChange(e: React.ChangeEvent<HTMLInputElement>) {
        setForm({
        ...form,
        [e.target.name]: e.target.value,
        });
    }

    return (
        <form onSubmit={handleSubmit}>
        <input
            name="name"
            placeholder="Nom"
            value={form.name}
            onChange={handleChange}
        />

        <input
            name="email"
            type="email"
            placeholder="Email"
            value={form.email}
            onChange={handleChange}
        />

        <input
            name="password"
            type="password"
            placeholder="Mot de passe"
            value={form.password}
            onChange={handleChange}
        />

        <CheckPassword
            password={form.password}
            onValidityChange={setIsPasswordValid}
        />

        <button type="submit" disabled={!isFormValid}>
            S'inscrire
        </button>
        </form>
    );
```

```
import { useState } from "react";  4.6k (gzipped: 1.9k)
import { useAppSelector } from "../../store/store";
import { UpdateProfile } from "../../updateProfile/components/UpdateProfile";

Windsurf: Refactor | Explain | Generate JSDoc | X
export const Profile = () => {
    const user = useAppSelector((state) => state.auth.user);
    const [editing, setEditing] = useState(false);

    if (!user) {
        return <p>You are not logged in</p>;
    }

    return (
        <div>
            <h1>{user.name}</h1>
            <p>{user.email}</p>

            {!editing && (
                <button onClick={() => setEditing(true)}>
                    Edit profile
                </button>
            )}

            {editing && (
                <UpdateProfile
                    user={user}
                    onDone={() => setEditing(false)}
                />
            )}
        </div>
    );
};
```

```
import { useState } from "react";  4.6k (gzipped: 1.9k)
import { useAppDispatch } from "../../store/store";
import { updateUser } from "../../store/auth/auth.slice";
import type { User } from "../../model/user";

Windsurf: Refactor | Explain
interface Props {
    user: User;
    onDone: () => void;
}

Windsurf: Refactor | Explain | Generate JSDoc | X
export const UpdateProfile = ({ user, onDone }: Props) => {
    const dispatch = useAppDispatch();

    const [name, setName] = useState(user.name);
    const [email, setEmail] = useState(user.email);
    const [message, setMessage] = useState("");

    Windsurf: Refactor | Explain | Generate JSDoc | X
    const handleSubmit = (e: React.FormEvent) => {
        e.preventDefault();

        dispatch(
            updateUser({
                ...user,
                name,
                email,
            })
        );

        setMessage("Profile successfully updated");
        onDone();
    };
```

```
    return (
        <form onSubmit={handleSubmit}>
            <div>
                <label>Name</label>
                <input
                    value={name}
                    onChange={(e) => setName(e.target.value)}
                />
            </div>

            <div>
                <label>Email</label>
                <input
                    value={email}
                    onChange={(e) => setEmail(e.target.value)}
                />
            </div>

            <button type="submit">Update</button>

            {message && <p>{message}</p>}
        </form>
    );
};
```

```
import { useLogin } from "../hook/use.login.hook";
    You, 13 hours ago • Reset Structure for 3WA, mixing feature base
Windsurf: Refactor | Explain | Generate JSDoc | X
export const Login = () => {
    const { login, user, error } = useLogin();
    const [email, setEmail] = useState("");
    const [password, setPassword] = useState("");

    Windsurf: Refactor | Explain | Generate JSDoc | X
    const onSubmit = (e: React.FormEvent) => {
        e.preventDefault();
        login(email, password);
    };

    if (user) {
        return <Navigate to="/profile" replace />;
    }

    return (
        <form onSubmit={onSubmit}>
            <label htmlFor="email">Email</label>
            <input
                aria-label="email"
                id="email"
                type="email"
                value={email}
                onChange={(e) => setEmail(e.target.value)}
            />

            <label htmlFor="password">Mot de passe</label>
            <input
                aria-label="password"
                id="password"
                type="password"
                value={password}
                onChange={(e) => setPassword(e.target.value)}
            />

        {error && <p>{error}</p>}

            <button type="submit">Login</button>
        </form>
    );
};
```

```
You, 2 hours ago | 1 author (You)
import { useSelector } from "react-redux";  2.2k (gzipped: 1.1k)
import type { AppState } from "../../store/store";
import { useAppDispatch } from "../../store/store";
import { loginRequest } from "../../store/auth/auth.slice";

Windsurf: Refactor | Explain | Generate JSDoc | X
export const useLogin = () => {
  const dispatch = useAppDispatch();
  const { user, error } = useSelector((state: AppState) => state.auth);

  Windsurf: Refactor | Explain | Generate JSDoc | X
  const login = (email: string, password: string) => {
    dispatch(loginRequest({ email, password }));
  };

  return {
    user,
    error,
    login,
  };
};
```

```
interface CheckPasswordProps {
    password: string;
    onValidityChange: (isValid: boolean) => void;
}

Windsurf: Refactor | Explain | Generate JSDoc | X
export const CheckPassword: React.FC<CheckPasswordProps> = ({
    password,
    onValidityChange,
    }) => {
    const rules = {
        length: password.length >= 12,
        uppercase: /[A-Z]/.test(password),
        lowercase: /[a-z]/.test(password),
        number: /[0-9]/.test(password),
        special: /[^A-Za-z0-9]/.test(password),
    };

    const isValid = Object.values(rules).every(Boolean);

    React.useEffect(() => {
        onValidityChange(isValid);
    }, [isValid, onValidityChange]);

    Windsurf: Refactor | Explain | Generate JSDoc | X
    function renderRule(label: string, valid: boolean) {
        return (
        <li style={{ color: valid ? "green" : "red" }}>
            {valid ? "✓" : "✗"} {label}
        </li>
        );
```

```
    }) => {
    const rules = {
        length: password.length >= 12,
        uppercase: /[A-Z]/.test(password),
        lowercase: /[a-z]/.test(password),
        number: /[0-9]/.test(password),
        special: /[^A-Za-z0-9]/.test(password),
    };

    const isValid = Object.values(rules).every(Boolean);

    React.useEffect(() => {
        onValidityChange(isValid);
    }, [isValid, onValidityChange]);

    Windsurf: Refactor | Explain | Generate JSDoc | X
    function renderRule(label: string, valid: boolean) {
        return (
        <li style={{ color: valid ? "green" : "red" }}>
            {valid ? "✓" : "✗"} {label}
        </li>
        );
    }

    return (
        <ul>
        {renderRule("12 caractères minimum", rules.length)}
        {renderRule("Une majuscule", rules.uppercase)}
        {renderRule("Une minuscule", rules.lowercase)}
        {renderRule("Un chiffre", rules.number)}
        {renderRule("Un caractère spécial", rules.special)}
        </ul>
    );
};
```

Custom Hooks

- Gère le dispatch pour le login, on envoie les données dans le store

```
You, 3 hours ago | 1 author (You)
import { useSelector } from "react-redux";  2.2k (gzipped: 1.1k)
import type { AppState } from "../../store/store";
import { useAppDispatch } from "../../store/store";
import { loginRequest } from "../../store/auth/auth.slice";

Windsurf: Refactor | Explain | Generate JSDoc | X
export const useLogin = () => {
  const dispatch = useAppDispatch();
  const { user, error } = useSelector((state: AppState) => state.auth);

  Windsurf: Refactor | Explain | Generate JSDoc | X
  const login = (email: string, password: string) => {
    dispatch(loginRequest({ email, password }));
  };

  return {
    user,
    error,
    login,
  };
};
```

Tests unitaires

```
PS D:\3WA\2025\Projet_Final\event-hub\eventhub_frontend> npm test

> eventhub_frontend@0.0.0 test
> jest

ts-jest[config] (WARN) message TS151001: If you have issues related to impo
). See https://blogs.msdn.microsoft.com/typescript/2018/01/31/announcing-ty
ts-jest[config] (WARN) message TS151001: If you have issues related to impo
). See https://blogs.msdn.microsoft.com/typescript/2018/01/31/announcing-ty
ts-jest[config] (WARN) message TS151001: If you have issues related to impo
). See https://blogs.msdn.microsoft.com/typescript/2018/01/31/announcing-ty
ts-jest[config] (WARN) message TS151001: If you have issues related to impo
). See https://blogs.msdn.microsoft.com/typescript/2018/01/31/announcing-ty
 PASS  src/modules/checkPassword/tests/CheckPassword.test.tsx
 PASS  src/modules/profile/tests/Profile.test.tsx
 PASS  src/modules/login/tests/Login.test.tsx
 PASS  src/modules/register/tests/Register.test.tsx

Test Suites: 4 passed, 4 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        1.931 s, estimated 2 s
Ran all test suites.
PS D:\3WA\2025\Projet_Final\event-hub\eventhub_frontend>
```

Démonstration en direct

Difficultés et solutions

- Comment accéder à store ?
- Comment respecter l'architecture / SOLID / Partiellement le POO ?

Solutions :

Merci !

Lien Github :
https://github.com/xdam99/event-hub/tree/dev